CAN SYSTEM ONE ROUTING REDUCE THE COST OF A CODING AGENT?
CS329Z MIDWAY REPORT DRAFT

Kristjan Dagur Egilsson **Ómar Ingi Halldórsson** **Rami Ratl Mrad**

1 RESEARCH QUESTIONS

Our project asks whether a small, structured decision model can identify coding tasks that a cheaper coding agent can solve, reducing inference cost while preserving the success rate of a stronger reference agent. We study a fixed OpenCode harness and change only the model-selection decision at task launch. Jev estimates whether the cheap model can complete the repair; a threshold determines whether the harness uses that model or a strong alternative. The primary intended comparison is task success versus total billed cost, including routing overhead. A secondary question concerns selective risk: among tasks accepted for the cheap model, how often does the resulting patch fail independent evaluation? The midpoint goal is a runnable system and auditable evaluation path; this constructed-task pilot supports engineering feasibility only.

2 ENVIRONMENT & DATA

Execution environment. The prototype wraps the existing OpenCode 1.19.5 command-line harness without replacing its agent loop. An orchestration script creates a fresh task directory containing a broken Python implementation and public smoke tests. OpenCode can inspect files, edit the designated implementation, and execute the smoke-test command. It produces an ordinary repaired source file, rather than a model-generated success label. The evaluator subsequently copies that file into a fresh directory and executes additional tests outside the agent workspace. Thus, patch correctness is determined by code execution. The current implementation runs on macOS with Python's standard library and an official, integrity-checked OpenCode binary. Docker was unavailable on the development machine, so repository-scale SWE-bench execution remains future work.

Prototype tasks and construction. We constructed six small repair fixtures, each specifying expected behavior and edge cases. Three development tasks cover stable deduplication, iterable chunking, and interval merging. Their independent graders contain six, seven, and eight tests, respectively. Tests include input-order preservation, generators, invalid arguments, nested intervals, and input mutation. The other three fixtures concern whitespace normalization, rolling means, and nested-list flattening. They are reserved for a later sanity check and have not been run through either coding model. These are inexpensive integration fixtures, not representative software engineering tasks.

Each fixture has a broken starter, a reference implementation used only to validate the grader, public example tests, and independent evaluation tests. Before model runs, we checked that every starter fails and every reference passes its independent grader. We also verified that terminating the evaluator process successfully without executing its tests is rejected. Public examples are available to the coding agent, but the independent test source and reference implementation are excluded from the router request and task directory. This separation guards against accidental leakage in the prototype; it does not establish resistance to malicious code or all forms of test gaming.

Evaluation protocol. Our initial comparison contains one rollout for each of three development tasks under three policies: always cheap, always strong, and Jev-gated. All nine post-integration runs use the same source revision, task specifications, tools, output limit, and step limit. Baseline runs precede routed runs. We preserve unsuccessful setup attempts separately and include their billed usage in session spending. The lower-threshold demonstration and fresh-source reproduction are supplementary, not independent primary observations.

For the final study, we intend to use a preselected, affordable subset of SWE-bench Lite with its official evaluator, following the task-level evaluation setting of Jimenez et al. (2024). We will freeze data revisions, task IDs, routing configuration, resource limits, and analysis rules before inspecting final outcomes. The official development set is small, so collecting sufficient independent calibration data remains an explicit concern. Public benchmark exposure during model pretraining also limits contamination claims.

Success criteria and measurements. A prototype run passes only if the independently tested patch passes, the harness exits successfully without timing out, and at least one model request was recorded. We record the selected model, actual upstream request model, provider, tool events, test outcome, end-to-end elapsed time, and billed API cost. For the final investigation, we will report task success, dollars per task and per successful task, latency, cheap-model coverage, and failure among cheap-accepted tasks. The intended five-percent selective-risk target is separate from a five-percentage-point noninferiority margin relative to the strong baseline. Neither target is established by the midpoint sample.

3 METHODS

One bounded routing decision. The router receives the issue specification, broken starter file, chosen model pair, and a description of the permitted tools and resource limits. It does not receive reference code, independent tests, or observed baseline outcomes. Jev answers one yes/no question about whether the cheap coding setup can solve that repair within the fixed environment. We retain the raw probability and full response (OpenRouter, 2026; TypeSafe AI, 2026). With provisional threshold $\tau = 0.90$, the routing rule is

$$m(x) = \begin{cases} m_{\text{cheap}}, & q(x) \geq \tau, \\ m_{\text{strong}}, & q(x) < \tau \text{ or the gate fails.} \end{cases}$$

The threshold was fixed before the development pilot and has not been calibrated. A timeout, malformed probability, or unavailable gate routes to the strong model and retains the failed request's cost reservation. This is pre-execution selection; the primary policy does not run the cheap model first and retry a failed repair with the strong model.

Model pair and harness controls. Following OpenCode configuration guidance (OpenCode contributors, 2026), the cheap coding model is `qwen/qwen3-coder-next`; the strong reference is `openai/gpt-6.1-sol`; the gate is `typesafe/jev-1.14`. All are accessed through OpenRouter. The coding harness has eight agent steps, a 2,048-token output cap per model request, at most ten requests, and a 180-second process timeout. Both coding conditions use the same permission configuration and task inputs. The observed coding providers in the primary pilot were Parasail and OpenAI, respectively. Provider fallback is disabled for each request, but provider selection itself is not yet frozen across the full study; actual provider metadata is retained for analysis.

Metering and budget enforcement. A local HTTP adapter sits between OpenCode and OpenRouter. It checks that the request uses the selected model, bounds request size and output tokens, captures usage costs, and forwards the response to the unchanged harness. The real upstream credential remains in the orchestrator; the child harness receives a temporary local-adapter token. Before a request, a durable ledger reserves a conservative cost using input bytes, message overhead, and the dated model price snapshot. Billed usage settles that reservation when available; otherwise the reservation remains charged. The prototype enforces a $0.50 task cap and a $5 session cap. The ledger is shared across runs and protected against concurrent writers. These controls are intended to make small experiments predictable, not to replace provider billing records.

Reproducibility and failure accounting. Each run preserves an immutable manifest with task and source hashes, protocol settings, route, raw event trace, request receipts, patch, and independent evaluation result. Reusing a completed run ID under the same protocol returns its stored result without another paid call; changing the protocol under that ID is rejected. Incomplete runs remain available for inspection. Eleven offline checks exercise the grader, leakage boundaries, invalid-probability rejection, routing fallback, reservation recovery, request caps, reserved task access, and

Table 1: Synthetic development pilot. Each row contains the same three tasks, one rollout per task. Cost includes all captured model calls and the gate where applicable. These are engineering checks, not benchmark results.

Policy	Passed	Mean cost (USD)	Mean time (s)
Always cheap	3/3	0.002593	9.39
Always strong	3/3	0.017650	11.48
Jev gate ($\tau = 0.90$)	3/3	0.018397	12.11

completed-run reuse. We also executed the installation, checks, and a real cheap-model repair from a fresh source copy. An independent teammate reproduction is still needed.

Planned calibration and comparisons. A larger development set must expose variation in cheap-model success before calibration is informative. We will assess score reliability and select a threshold on development data, then lock evaluation. Always-cheap and always-strong policies will be supplemented by deterministic heuristic routing and random routing at matched cheap coverage, with cost-matched sensitivity analysis. We will report paired uncertainty and high-confidence failures. Tinker training requires sufficient data and a diagnosed routing weakness.

4 Summary of Progress

Implemented and measured. We have implemented the one-command routing path, real OpenCode execution, separate patch evaluation, request-level metering, persistent manifests, and bounded/resumable runs. Table 1 summarizes the nine post-integration development rollouts recorded on November 3. Every policy passed all three constructed tasks. The initial Jev scores were 0.79, 0.59, and 0.58; at $\tau = 0.90$ all three tasks used the strong model. Cheap coverage was therefore zero, and selective risk is undefined rather than zero. Mean gate latency was approximately 0.245 seconds.

The pilot does not demonstrate a routing advantage. In particular, the always-cheap baseline already solved all three tasks, while the gate selected the stronger model every time. Differences between the always-strong and routed bills reflect both the extra gate call and different stochastic coding trajectories; we cannot attribute the full difference to routing overhead. The fixtures are too easy and too few to evaluate discrimination, establish equivalence, or estimate a useful cost/performance frontier.

Engineering obstacles and supplementary checks. An initial sandbox restriction prevented the local adapter from binding a socket. A subsequent attempt exposed an effective-working-directory/path-permission mismatch: the harness could inspect files but failed to apply the intended repair. We preserved that failed attempt and corrected the directory invocation and path rules before the comparison. A separately labeled, post-hoc demonstration uses $\tau = 0.75$ on the deduplication development task, routing its score of 0.79 to the cheap model and passing the independent grader. This verifies the cheap-routing execution path; it is not calibrated threshold selection or held-out evidence.

Remaining work and risks. Before midpoint submission, the team must review this draft and its references, reproduce one task independently using the README, verify grader repository access, and submit the report, repository link, and recorded video. The supplied replay material must be described as a replay of recorded execution, rather than live execution. For the final study, the main risks are establishing the container environment, obtaining enough development/calibration tasks, and selecting tasks that expose meaningful differences between models. We will prioritize paired evaluation and error analysis over adding models. Risk guarantees require more independent accepted cases. The final paper will add explicit safety and authorship statements.

REFERENCES

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations*. https://arxiv.org/abs/2310.06770.

OpenCode contributors. 2026. OpenCode documentation: command-line interface, agents, and permissions. Software documentation. https://opencode.ai/docs/cli/. Accessed November 3, 2026.

OpenRouter. 2026. Jev documentation: using the TypeSafe decision model on OpenRouter. API documentation. https://openrouter.ai/docs/guides/community/jev. Accessed November 3, 2026.

TypeSafe AI. 2026. Introduction and confidence. Model documentation. https://docs.typesafe.ai/introduction; https://docs.typesafe.ai/confidence. Accessed November 3, 2026.

AI-USE DISCLOSURE STATEMENT

Rami Ratl Mrad used Codex to interpret the proposal and instructor feedback, develop the execution plan, implement and debug the prototype and offline checks, construct synthetic repair fixtures and reference/evaluation code, run the configured model experiments, aggregate their recorded outputs, and draft this report and demonstration material. Jev supplied routing probabilities, and the named coding models generated repairs during evaluation. Codex-generated tests are not an independent source of benchmark validity; the team must review their specifications and correctness. This draft does not claim that other team members have already reviewed or reproduced these artifacts. The authors are responsible for checking all code, measurements, references, and submitted text.